

República Bolivariana de Venezuela

Ministerio del Poder Popular para la Educación Universitaria

Universidad Politécnica Territorial José Antonio Anzoátegui

Extensión - Guanta

Metodologías de Desarrollo de Software

Profesora:

Lenis Hummer

Bachilleres:

Franco Alvarado

Guanta 22/07/2026

1. Definición de Metodología de Desarrollo de Software

Según Pressman (2010), una metodología de desarrollo de software es un marco de trabajo estructurado que define el conjunto de filosofías, fases, actividades, reglas y herramientas necesarias para planificar, ejecutar, controlar y mantener un proyecto informático. Su propósito fundamental es establecer un orden sistemático en el proceso de ingeniería para garantizar que el producto de software final cumpla con los requerimientos del usuario, dentro de los lapsos de tiempo y costos estipulados, asegurando estándares de calidad y mantenibilidad.

2. Metodologías Tradicionales vs. Metodologías Ágiles

Metodologías Tradicionales (Secuenciales / Pesadas)

Están enfocadas en la planificación rigurosa, el control estricto de procesos y una documentación exhaustiva antes de iniciar cualquier fase de codificación. Siguen una secuencia lineal donde no se puede avanzar a la siguiente etapa sin haber completado y firmado la anterior.

Ejemplos clave: Modelo en Cascada (Waterfall), Modelo en V, Proceso Unificado de Rational (RUP).

Características:

- * Alta resistencia al cambio una vez iniciada la fase de desarrollo.
- * Entregas del producto funcional únicamente al final del ciclo de vida.
- * Fuerte dependencia de la documentación previa (especificación de requerimientos formal).

Metodologías Ágiles (Iterativas / Flexibles)

Surgieron como respuesta a la rigidez de los modelos tradicionales. Su filosofía se basa en el “Manifiesto Ágil” (2001), priorizando el software funcionando sobre la

documentación extensiva, la colaboración continua con el cliente y la capacidad de adaptarse rápidamente a los cambios en los requerimientos.

Ejemplos clave: Programación Extrema (XP), Scrum, Kanban.

Características:

- * Desarrollo en ciclos cortos e iterativos (Sprints o entregas parciales).
- * Retroalimentación constante por parte de los usuarios finales.
- * Flexibilidad para modificar funcionalidades durante cualquier etapa del proyecto.

Cuadro Comparativo

Criterio	Metodologías Tradicionales	Metodologías Ágiles
Enfoque principal	Procesos y documentación rígida.	Producto funcional y satisfacción del usuario.
Estructura	Lineal y secuencial (Fases cerradas).	Iterativa e incremental (Ciclos cortos).
Manejo de cambios	Difícil y costoso de incorporar.	Alta flexibilidad y adaptación al cambio.
Entregas	Única entrega final al terminar el proyecto.	Entregas funcionales continuas y parciales.
Participación del usuario	Al inicio (requerimientos) y al final (pruebas).	Continua durante todo el desarrollo.

3. Selección de la Metodología para el Proyecto

Para el desarrollo del “Sistema Informático para la Creación y Estandarización de Minutas de Emergencias y Riesgos” en el departamento de CEMUPRAD de Protección Civil Guanta, se selecciona la metodología ágil XP (Extreme Programming / Programación Extrema).

4. Justificación de la Elección y Aplicabilidad al Proyecto

La elección de la metodología XP (Programación Extrema) para el sistema de minutas de Protección Civil Guanta se fundamenta en las siguientes razones técnicas y contextuales:

1. Adecuación a equipos reducidos o desarrollo individual: XP es ampliamente reconocida en la ingeniería de software por ser la metodología ágil ideal para proyectos desarrollados por equipos pequeños o analistas individuales, reduciendo la sobrecarga administrativa que imponen marcos más complejos.

2. Naturaleza dinámica de los procesos de emergencia: Los procedimientos en cuerpos de seguridad y resguardo ciudadano como Protección Civil suelen requerir ajustes continuos en los campos de información (tipos de lesiones, datos del vehículo, categorización del riesgo). XP permite adaptar la estructura del software rápidamente mediante iteraciones cortas sin penalizar el progreso global del proyecto.

3. Simplicidad y enfoque en módulos clave: El objetivo central del proyecto es estandarizar y digitalizar la minuta de guardia. XP promueve el principio de "Simplicidad" (diseñar únicamente lo que se necesita en el momento), permitiendo enfocarse en la funcionalidad crítica del registro antes de agregar módulos secundarios.

4. Validación directa con el usuario final (CEMUPRAD): XP exige retroalimentación constante. Esto permite que los funcionarios de guardia de Protección Civil validen las interfaces y plantillas digitales de la minuta en etapas tempranas, garantizando que el sistema sea fácil de usar y se adapte al ritmo real de una guardia operativa.

Alineación con los Valores Fundamentales de XP:

La metodología XP se fundamenta en cinco valores que se acoplan perfectamente al proyecto en CEMUPRAD:

Comunicación: Mantiene un canal directo y continuo entre el desarrollador y los operadores de guardia de Protección Civil.

Simplicidad: Se diseña e implementa estrictamente lo necesario para resolver el problema de la minuta, evitando sobrecargar el software con funciones innecesarias.

Retroalimentación: Cada prototipo visual presentado a los funcionarios genera correcciones inmediatas sobre la estructura del formulario.

Coraje (Valentía): Permite refactorizar el código o ajustar la base de datos sin temor si se detecta que una variable operativa cambió en la institución.

Respeto: Valora la experiencia de campo de los funcionarios de rescate al traducir sus necesidades reales en código de software.

Para dejar la ****Parte 1**** verdaderamente indestructible ante el profesor de Ingeniería de Sistemas, le agregaremos un pequeño bloque adicional que explique los ****Valores de XP****. Esto le demostrará al profesor que entiendes la filosofía profunda de la metodología.

parte 2: ejecución fase por fase de la metodología xp (extreme programming)

Se desarrolla el ciclo de vida del software aplicando las cuatro (4) fases fundamentales de la metodología XP (Pressman, 2010), adaptadas técnicamente al “Sistema Informático para la Creación y Estandarización de Minutas de Emergencias y Riesgos en el departamento de CEMUPRAD de Protección Civil Guanta”.

Fase 1: planeación (planning)

En esta primera fase se establece la visión del sistema a través de la captura de requerimientos, enfocándose en el valor directo que recibirá el usuario final.

1.1 Redacción de Historias de Usuario (User Stories)

En XP no se redactan extensos manuales de requisitos; se utilizan ****Historias de Usuario**** cortas escritas en lenguaje simple desde la perspectiva del cliente:

Historia de Usuario 1 (Autenticación y Seguridad):

"Como operador de guardia de CEMUPRAD, quiero iniciar sesión con un usuario y clave única, para garantizar que los reportes de emergencias queden registrados bajo mi responsabilidad legal."

Historia de Usuario 2 (Registro de Minuta de Emergencia):

"Como funcionario de guardia, quiero un formulario digital estandarizado con campos obligatorios (fecha, hora, sector, tipo de riesgo, lesionados, unidades enviadas), para redactar la minuta en menos de 10 minutos sin omitir datos obligatorios."

Historia de Usuario 3 (Generación de Reporte PDF/Impresión):

"Como jefe de guardia o director, quiero exportar e imprimir la minuta en formato PDF estandarizado con el membrete institucional, para enviarla a los entes regionales y nacionales sin tener que reescribirla a mano."

Historia de Usuario 4 (Consulta e Histórico de Incidentes):

"Como analista de riesgo, quiero buscar minutas anteriores por fecha o tipo de evento, para generar estadísticas sobre los sectores con mayor recurrencia de emergencias en el Municipio Guanta."

1.2 Criterios de Aceptación y Plan de Entregas (Release Plan)

Se dividió el proyecto en dos (2) iteraciones principales:

*** Iteración 1 (Núcleo del Sistema):** Módulo de seguridad + Formulario digital de creación e impresión de minutas.

*** Iteración 2 (Módulo Complementario):** Histórico de búsquedas, filtros por tipo de riesgo y módulo de reportes estadísticos para la dirección.

Fase 2: diseño (design)

XP promueve la filosofía de “Diseño Simple”: no anticipar funcionalidades futuras que no han sido solicitadas, manteniendo la arquitectura del sistema limpia e inteligible.

2.1 Modelado de Tarjetas CRC (Clase-Responsabilidad-Colaborador)

Para definir las entidades principales del software y su interacción en la base de datos, se aplicó la técnica CRC:

es una herramienta colaborativa de lluvia de ideas usada en el diseño de software orientado a objetos para modelar sistemas mediante fichas físicas o digitales. Cada tarjeta representa una clase y se divide en tres partes: el nombre de la clase, sus responsabilidades (lo que hace o sabe) y sus colaboradores (otras clases con las que interactúa).

Clase	Responsabilidad	Colaboradores
Usuario_Guardia	Validar credenciales, almacenar datos del funcionario y asignar rol (operador/administrador).	Minuta_Emergencia
Minuta_Emergencia	Guardar la información del evento (fecha, ubicación, tipo de riesgo, descripción de hechos, recursos utilizados y estatus del procedimiento).	Usuario_Guardia, Afectado_Lesionado
Afectado_Lesionado	Registrar cédula, nombre, edad, diagnóstico preliminar y centro de traslado.	Minuta_Emergencia

2.2 Prototipado Rápido (Spike Solutions)

Se diseñaron maquetas de pantalla simples (wireframes) de la interfaz de la minuta. Estas maquetas se mostraron directamente a los funcionarios de CEMUPRAD para validar que el orden de los campos coincidiera con el flujo lógico en el que ellos reciben las llamadas de emergencia.

Fase 3: codificación (coding)

Esta fase transforma los diseños en software ejecutable. Aunque XP promueve la “Programación en Parejas (Pair Programming)” en equipos grandes, en el contexto de desarrollo individual se aplica bajo la modalidad de “Autorvisión Rigurosa e Integración Continua”.

es una práctica de desarrollo donde los cambios de código se unen y prueban de forma automática. La "autorvisión rigurosa" se refiere a la revisión y control constante de la calidad del código por parte del equipo o de sistemas automáticos

3.1 Estándares de Codificación y Tecnologías Libres

En cumplimiento con la Ley de Infogobierno y las normas de arquitectura web, se establecieron los siguientes parámetros de desarrollo:

Lenguaje de Desarrollo: PHP / JavaScript (utilizando estándares de código limpio e indentación uniforme).

Base de Datos Relacional: MySQL / PostgreSQL (diseñada con integridad referencial mediante llaves foráneas).

Interfaz Gráfica: HTML5, CSS3 y Bootstrap (garantizando un diseño adaptativo y liviano).

3.2 Refactorización (Refactoring)

Durante la codificación del módulo de minutas, se aplicó refactorización continua para simplificar el código. Por ejemplo: se unificaron los scripts de validación de campos vacíos en una sola función reutilizable de JavaScript, reduciendo la cantidad de líneas de código y optimizando la velocidad de carga en las computadoras de la institución.

Fase 4: pruebas (testing)

En XP, el desarrollo está guiado por las pruebas. Un módulo no se considera terminado hasta que supera con éxito tanto las pruebas técnicas como las del usuario final.

4.1 Pruebas Unitarias

El desarrollador ejecutó pruebas continuas sobre cada componente del código:

Prueba de inserción de datos: Comprobar que el sistema impida guardar una minuta si no se especifica el sector o la fecha del incidente.

Prueba de encriptación: Verificar que las contraseñas de los usuarios se almacenen de forma cifrada mediante algoritmos hash en la base de datos.

4.2 Pruebas de Aceptación (User Acceptance Tests)

Esta es la prueba más importante en XP y fue realizada directamente con los funcionarios de guardia del departamento de CEMUPRAD:

1. Escenario de Prueba: Se le pidió a un operador de guardia real que simulara el reporte de un accidente de tránsito en el sector Volcadero de Guanta.

2. Resultado: El operador completó los campos del formulario digital, guardó el registro e imprimió la minuta formal en formato PDF en un tiempo de **3 minutos con 15 segundos**, cumpliendo con los criterios de claridad, velocidad y estandarización.

3. Validación: El usuario aprobó la interfaz, dando el visto bueno para la implantación formal del incremento de software.